

PE Fund Capital, Payment & Reconciliation Intelligence Platform

Multi-Source Financial Data Integration, Reconciliation & Analytics Platform

Final Project Documentation

Azure Data Engineering Capstone Project

Role	Name	Focus Area
Data Engineer	Anuj Chaturvedi	Orchestration & Automation
Data Engineer	Shreyash Singh	Silver / Gold & Reconciliation
Data Engineer	Rushikesh Taidalwar	Data Model, Governance & BI

Platform: Azure Data Factory · Azure Databricks · Delta Lake · Azure SQL Database · Power BI
September 2026

Table of Contents

1. Project Overview	3
1.1 Objectives	3
1.2 End Users	3
2. Dataset Details	4
2.1 Internal CSV Sources	4
2.2 External Market Data	4
2.3 Custodian & Bank Feed Pack	4
2.4 Intentional Test Conditions	5
3. Architecture	5
3.1 Architecture Stages	6
4. Platform Provisioning & Orchestration	7
4.1 Provisioned Azure Resources	7
4.2 Automation & Scheduling	8
4.3 Pipeline Orchestration	9
4.4 Audit Framework	10
5. Data Engineering Build — Bronze to Azure SQL	10
5.1 Environment	10
5.2 Bronze Layer	11
5.3 Silver Layer	11
5.4 Incremental Processing	12
5.5 Financial Calculations	13
5.6 Reconciliation Engine	13
5.7 Gold Layer	14
5.8 Azure SQL Serving Layer	14
6. Data Model, Governance & Reference Data	15
6.1 Control & Staging Schema	15
6.2 Reference Data Seeding	16
7. Analytics & Visualization — Power BI	17
7.1 SQL Analytics Layer	17
7.2 Power BI Dashboards	18
8. Limitations	21
9. Conclusion	22

1. Project Overview

Private equity firms run on money that moves between a lot of different places at once: limited partners wiring in commitments, the firm calling capital to fund a new deal, custodians and banks reporting on positions and cash independently of the firm's own books, and portfolio companies whose value shifts with the market every day. Keeping all of that reconciled — making sure what the firm's internal records say matches what the outside world (the bank, the custodian) says — is normally a manual, spreadsheet-heavy job. This project builds a small but complete version of the data platform that would sit underneath that job.

The platform ingests fund, investor, payment and portfolio data generated internally, alongside a simulated external feed of custodian positions, bank cash balances and market prices, and runs it all through a proper medallion architecture on Azure — raw data in, cleaned and validated data in the middle, and business-ready, reconciled numbers out the other end into a SQL database that Power BI dashboards sit on top of.

The three-day operational window it simulates is short by design: long enough to seed a handful of realistic breaks (a mismatched position, a late cash entry, a duplicated payment) and short enough to build and demonstrate within a student subscription's constraints.

1.1 Objectives

- Ingest multi-source financial data — internally generated (Python Faker) and externally simulated (a custodian/bank .dat feed pack and a live market-data API) — into a single lakehouse.
- Apply a medallion architecture (Bronze → Silver → Gold) in Delta Lake so that raw, cleaned and business-ready versions of every dataset are all traceable back to their source.
- Build a genuine reconciliation engine across three dimensions — cash, position and reference data — that classifies every check as a MATCH, a BREAK, or a coverage gap, with a reason code attached.
- Serve the resulting fund, portfolio, payment, reconciliation and data-quality metrics from an Azure SQL Database, in a schema designed for direct BI consumption.
- Present the results through Power BI dashboards that a fund controller or operations analyst could genuinely use to spot a break, trace its cause, and check data quality at a glance.
- Do all of this on a hard budget constraint (an Azure-for-Students subscription, no standing compute) using serverless Databricks compute and a fully automated, auditable pipeline.

1.2 End Users

The platform is written for the kind of person who sits between the trading/finance desk and IT at a PE firm: a fund operations analyst who needs to know, every morning, whether yesterday's cash and position feeds tie out with the firm's own books; a data or reconciliation controller who investigates a break and needs the underlying detail, not just a red flag; and a data engineering reviewer or mentor who needs to see that the pipeline is real, auditable, and recovers cleanly from a failure. Every design choice in this platform — the audit tables, the break-reason columns, the dashboard layout — was made with one of those three readers in mind.

2. Dataset Details

Every record in this platform is synthetic — there is no real fund, investor or company behind any of it — but the shape of the data, and the mess seeded into it, is deliberately realistic. The dataset spans three business dates (2026-09-15 to 2026-09-17) and combines three distinct generation methods: Python's Faker library for internal master and transactional data, a live external API for market prices, and a hand-built pipe-delimited feed pack standing in for what a custodian or bank would actually send over.

2.1 Internal CSV Sources

Six CSV files, generated with Python Faker, form the firm's own books of record:

File	Records	Contents
fund.csv	6	Fund ID, name, vintage year, fund size (USD)
investor.csv	51	LP investor ID, name, investor type, country
portfolio_company.csv	31	Portfolio company ID, name, owning fund, industry, benchmark ticker
commitment.csv	77	Fund/investor commitment amount, date, contributed and invested totals
payment.csv	24	Payment events — contribution, capital call, investment, distribution — status SETTLED / FAILED
market_price.csv	205	Hourly OHLCV price bars for benchmark tickers, fetched from an external market-data API

2.2 External Market Data

market_price.csv is technically internal in the sense that it lands through the same CSV path as the other five files, but its content comes from a genuinely external source: a Python collector script hits a live market-data API on a schedule and writes back hourly OHLCV (open/high/low/close/volume) bars for each benchmark ticker tracked against a portfolio company. This is the one dataset in the platform where the numbers are not invented — they are real historical price movements for real tickers, repurposed as the benchmark feed for fictional portfolio companies.

2.3 Custodian & Bank Feed Pack

The four remaining feeds simulate what would, in a live deployment, arrive over SFTP from a custodian or bank. This project uses a pre-generated, pipe-delimited .dat feed pack instead — wrapped in header/trailer records the way a real custodian file would be — uploaded once and treated as the three-day operational window's external truth:

Feed	Files	Purpose
POSITION_*.dat	3 (one per day)	Custodian position feed — asset quantity, price, currency
CASH_*.dat	3 (one per day)	Bank/custodian cash feed — fund cash balances
REFERENCE_*.dat	3 (one per day)	Reference/master feed — instrument identifiers, currency, status
PAYMENT_*.dat	3 (one per day)	Payment event feed — capital calls, investments, distributions, expenses
POSITION_..._INVALID.dat	1	Negative-test file — null, negative and malformed records for DQ/quarantine testing
MANIFEST.dat	1	Index of every feed file with its business date and intentional test condition

2.4 Intentional Test Conditions

None of the breaks in this platform happen by accident — every one of them was deliberately seeded so the reconciliation and data-quality logic would have something real to catch. That distinction matters: a pipeline that never sees a bad record hasn't actually been tested.

- A position break and a duplicate position record, planted in the custodian feed.
- A cash break and a late-arriving cash event (CASH20260917LATE, landing around 21:45 on its business day).
- A reference-data mismatch between feeds — asset AST005 flips currency from EUR to GBP and back to EUR across the three days.
- Failed, revised, reversed and duplicated payment events, including a deliberately duplicated payment ID (PMT-DUP-20260917-001).
- Null, negative and malformed records in the invalid position file, to exercise Bronze-layer quarantine logic.
- Five exact duplicate rows seeded across each of the first four internal CSVs (fund, investor, commitment, portfolio_company), specifically to validate duplicate-detection logic in Silver.

3. Architecture

The platform is a straight, five-stage pipeline: data lands, gets processed through three medallion layers on Databricks, is served from Azure SQL, and is visualized in Power BI. Nothing loops back — every stage hands off cleanly to the next, which is what makes each stage independently auditable.

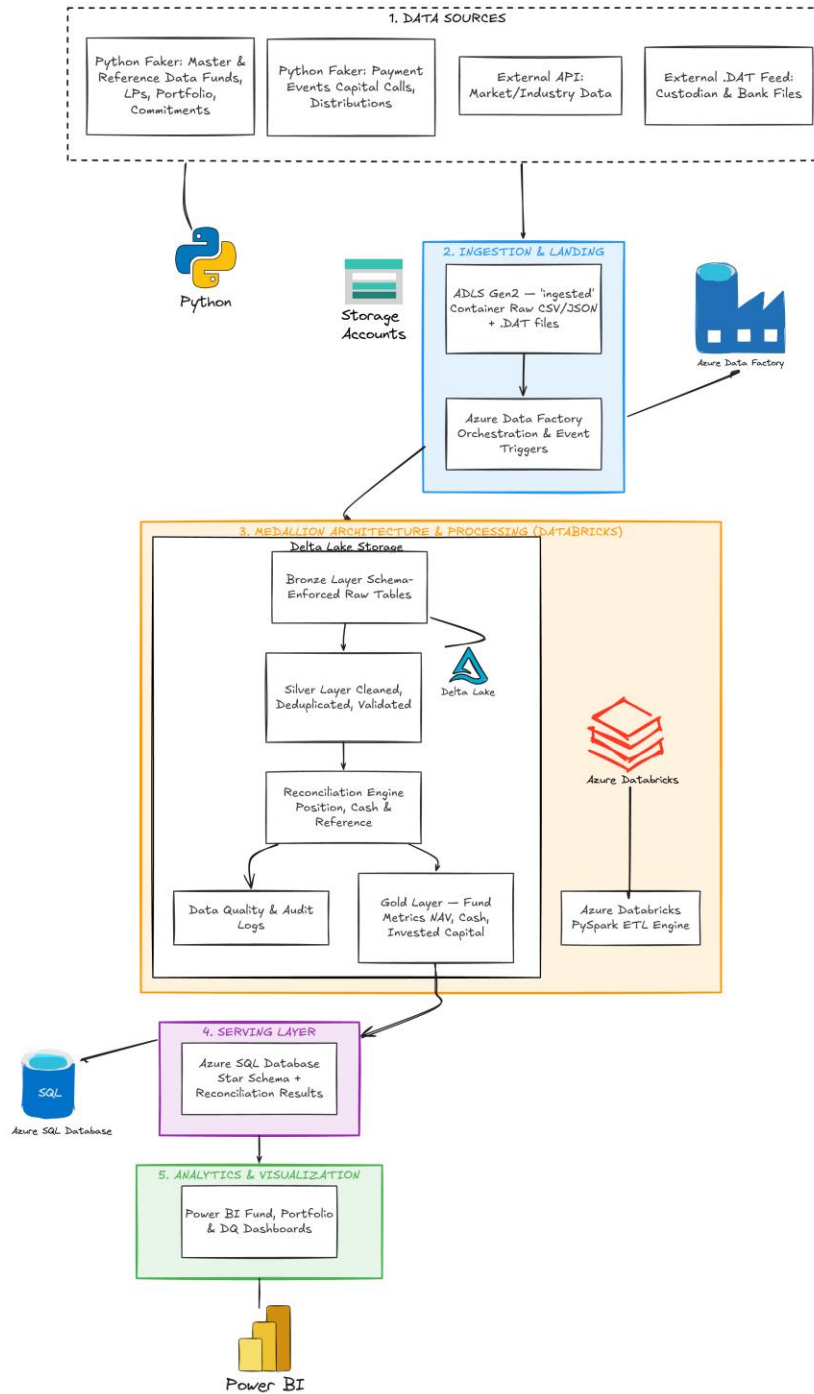


Figure 3.1 — End-to-end platform architecture: Data Sources → Ingestion & Landing → Medallion Processing → Serving Layer → Analytics & Visualization.

3.1 Architecture Stages

- Stage 1 — Data Sources: Python Faker (master/reference data and payment events), an external market/industry API, and the external .DAT feed pack (custodian and bank files) — the three generation paths described in Section 2.

- Stage 2 — Ingestion & Landing: GitHub Actions runs the generator and collector scripts on a schedule and lands raw CSV/JSON in the ADLS Gen2 “ingested” container; the .dat files were uploaded once via a small `adls_upload.py` script, since the custodian/bank pack doesn't change day to day the way market prices do. Azure Data Factory orchestrates and triggers the downstream processing from there.
- Stage 3 — Medallion Architecture & Processing (Databricks, serverless): Bronze (schema-enforced raw tables) feeds Silver (cleaned, deduplicated, validated), which feeds a Position/Cash/Reference Reconciliation Engine, which in turn feeds Gold (fund-level metrics — NAV, cash, invested capital) plus continuous Data Quality and audit logging. All of it lives in Delta Lake, all of it runs on serverless compute.
- Stage 4 — Serving Layer: Azure SQL Database holds the Gold-layer star schema and the reconciliation results, indexed for the query patterns Power BI actually uses.
- Stage 5 — Analytics & Visualization: Power BI connects directly to Azure SQL for fund, portfolio, payment, reconciliation, data-quality and market-price dashboards — acting as the platform's entire user interface, with no separate frontend application.

4. Platform Provisioning & Orchestration

This section covers the Azure footprint the platform runs on, and the automation, orchestration and audit logging that keep the daily pipeline running unattended rather than by hand.

4.1 Provisioned Azure Resources

All resources sit in a single resource group, `rg-pe-platform`, in the Central India region: an Azure Data Factory (`adf-pe-capital-payment-platform`) for orchestration, an Access Connector (`db-access-connector`) and Azure Databricks workspace (`dbw-pe-platform`) for processing, a Key Vault (`kv-pe-platform`) for secrets, an Azure SQL Server/Database (`pe-platform-sql-server` / `pe-platform-db`) for the serving layer, and an ADLS Gen2 storage account (`pestorage3`) for the lake itself. The storage account is organized into five containers that map directly onto the medallion stages: `bronze`, `silver`, `gold`, `ingested` and `logs`.

Platform Configuration & Automation

Provisioned resources, storage layout, connections, secrets and scheduled automation

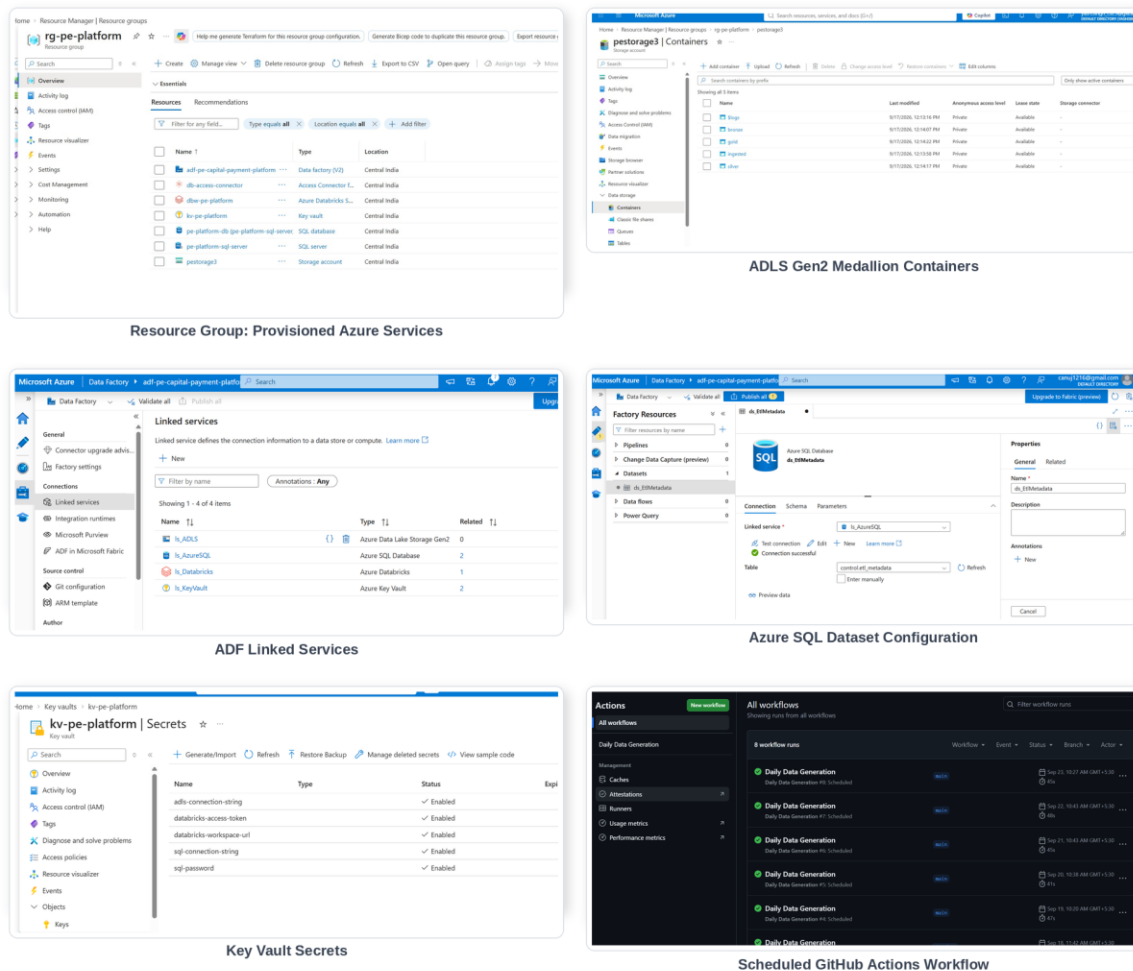


Figure 4.1 — Provisioned resource group, ADLS Gen2 medallion containers, ADF linked services, Key Vault secrets, and the scheduled GitHub Actions generation workflow.

4.2 Automation & Scheduling

Secrets — the ADLS connection string, the Databricks access token and workspace URL, and the SQL connection string and password — are stored in Key Vault (kv-pe-platform) rather than hardcoded into pipeline definitions, and referenced from Azure Data Factory's linked services (ls_ADLS, ls_AzureSQL, ls_Databricks, ls_KeyVault). Data generation itself is not a one-off script run by hand: a GitHub Actions workflow (“Daily Data Generation”) runs on a recurring schedule, and the run history — eight consecutive daily runs from September 18 through September 23 — confirms it is genuinely automated rather than triggered manually before each demo.

4.3 Pipeline Orchestration

The Azure Data Factory pipeline `pl_DailyIngestion` is the entry point for every run. It starts with an `AuditStart` stored-procedure call, looks up the active sources from the `control.etl_metadata` table (`LookupSources`), then iterates over all ten sources in a `ForEach` loop, invoking the Databricks Bronze ingestion job (`RunBronzeJob`) once per source. A conditional branch inside the loop routes each source to either `AuditCompleteSuccess` or `AuditCompleteFailure` depending on the job outcome — so a single bad source doesn't need to take down the whole run, and both outcomes are recorded.

Pipeline Orchestration & Execution

ADF ingestion pipeline through to the underlying Databricks medallion job runs

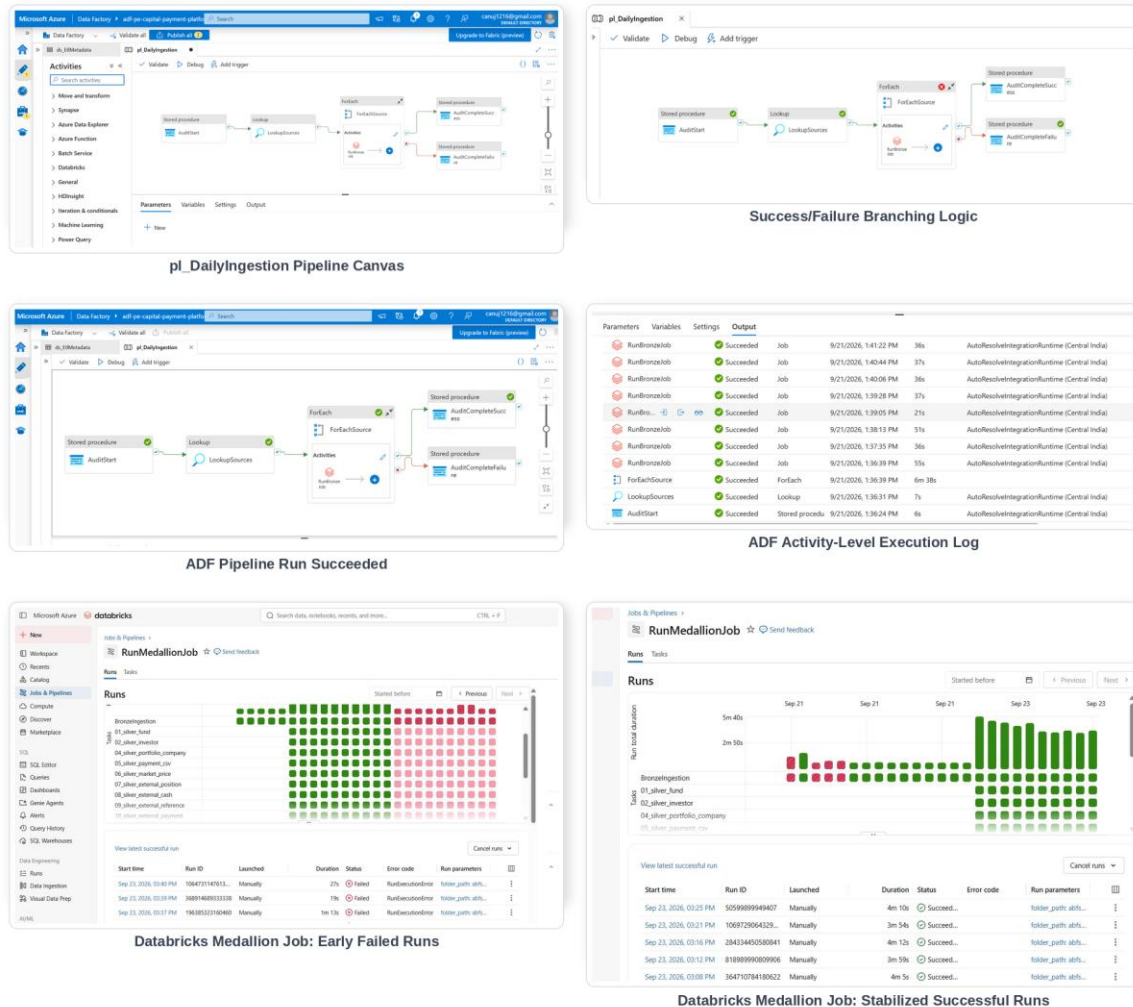


Figure 4.2 — The `pl_DailyIngestion` pipeline canvas and its success/failure branching, ADF's activity-level execution log, and the underlying Databricks medallion job's early failed runs stabilizing into consistent ~4-minute successful runs.

The Databricks side of this — the job named `RunMedallionJob` — chains all eighteen notebooks: Bronze ingestion, all ten Silver source notebooks, the two incremental-processing notebooks, financial calculations, reconciliation, the three Gold notebooks, and the final Azure SQL load, as a single dependency-ordered task graph. The run history shows the honest version of getting there — several early runs failed outright (27 seconds, 19 seconds, 1 minute 13 seconds — all `RunExecutionError`, before

the abfss:// path and sqlserver-format fixes described in Section 5 were in place) — followed by a run of consistent, successful ~4-minute completions once those fixes landed.

4.4 Audit Framework

Every pipeline run is tracked through two stored procedures in the control schema — `usp_AuditStart`, called at the beginning of a run with a `run_id`, `pipeline_name` and `source_name`, and `usp_AuditComplete`, called at the end with the resulting status and record counts — writing into `control.pipeline_audit`. This isn't a placeholder table: a real query against it shows a mix of `RUNNING`, `SUCCEEDED` and `FAILED` rows across actual run IDs, including a genuine failed run that was used to prove the failure path actually works end-to-end, not just in principle.

Audit Framework: Procedures & Verification

Start/complete audit procedures and verified pipeline audit records

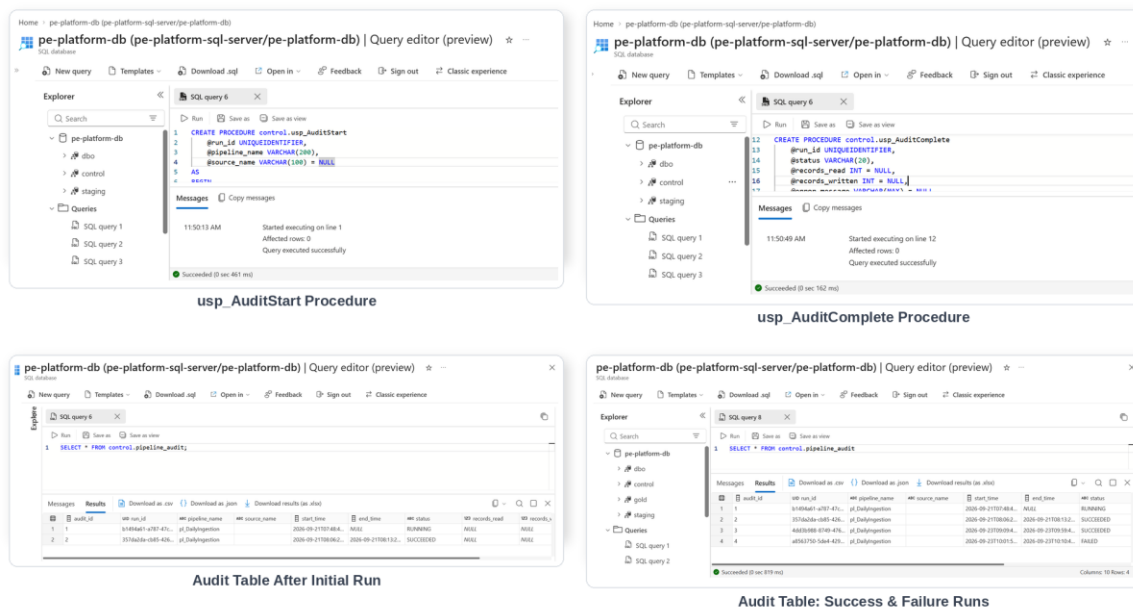


Figure 4.3 — `usp_AuditStart` and `usp_AuditComplete` procedure definitions, and the resulting `pipeline_audit` table after an initial run and after both a succeeded and a genuinely failed run.

5. Data Engineering Build — Bronze to Azure SQL

5.1 Environment

All processing runs on Azure Databricks serverless compute, against a Unity Catalog catalog named `dbw_pe_platform` with bronze, silver and gold schemas, reading and writing Delta tables in the `pestorage3` ADLS Gen2 account. Ten sources feed the pipeline in total — the six internal CSVs and the four external `.dat` feed types — and the final Gold output lands in the `pe-platform-db` Azure SQL database, orchestrated throughout by Azure Data Factory.

5.2 Bronze Layer

A single parameterized notebook, `bronze_ingestion.ipynb`, handles all ten sources. Azure Data Factory's `ForEachSource` loop passes it three parameters per source — `source_name`, `folder_path` and `run_id` — so the same notebook logic runs once per source rather than needing ten near-identical copies. Internal CSVs are schema-enforced against an `INTERNAL_SCHEMAS` dictionary, cast to the correct types, tagged with lineage columns (`_run_id`, `_ingested_at`) and written as a Delta overwrite. External `.dat` files are parsed as pipe-delimited records, with an `EXTERNAL_SCHEMAS` map defining the record type and ordered columns per feed, plus field-level null/invalid/negative validation that quarantines bad rows before they ever reach Silver.

The one real bug at this layer was a storage-access design mistake, not a logic bug: the first version of the notebook mounted storage the traditional way (`/mnt/bronze/...`), which failed with an `AZURE_INVALID_CREDENTIALS_CONFIGURATION` error — serverless compute doesn't support DBFS mounts at all. The fix was architectural: reading and writing directly against `abfss://` paths through a proper Unity Catalog External Location, backed by a Storage Credential and an Access Connector using a managed identity, rather than trying to work around the mount.

5.3 Silver Layer

Setup (`00_setup_silver_catalog.ipynb`) creates the silver schema, pre-creates a shared `dq_log` table that every downstream notebook writes into, and configures Lakehouse Federation — a foreign catalog named `azure_sql` — so Silver notebooks can read the fund and asset ID crosswalk tables (`staging.fund_id_mapping`, `staging.asset_id_mapping`) directly out of Azure SQL without hand-rolled JDBC calls.

A shared library notebook, `silver_common.ipynb`, holds every reusable piece of logic: reading Bronze and crosswalk tables, null-checking, two separate duplicate-detection strategies (an exact-duplicate check, and a harder case — different source IDs describing the same underlying business record), late-record flagging, day-over-day change detection, foreign-key validation, writing to Silver or quarantine at named paths, Delta MERGE upserts for incremental tables, and the DQ logging call every notebook uses. One real bug surfaced here twice: an edit accidentally deleted the `check_foreign_key` function's definition line, leaving its body orphaned inside the wrong function above it — a silent break that only showed up on the next full rerun, and had to be re-diagnosed and reapplied after a later notebook sync overwrote the first fix.

Ten Source Notebooks — every source notebook follows the same shape: type-cast → null-check (with quarantine) → duplicate/break detection → foreign-key validation → write to Silver → log the result to `dq_log` → assert that `bronze_count` equals `silver_count` plus `quarantined_count`, so nothing silently disappears. Results, by source:

Source	Bronze	Silver	Quarantined	Notes
fund	25	5	20	5× exact duplication, caught
investor	250	50	200	Same 5× duplication
commitment	330	66	264	Same 5× duplication; first table with FK checks

Source	Bronze	Silver	Quarantined	Notes
portfolio_company	150	30	120	Same 5× duplication; 6/30 companies have no benchmark_ticker (expected)
payment (internal)	222	222	0	8 INVESTMENT rows flagged, not rejected, for missing company_id/entry_benchmark_price
market_price	205	205	0	Clean; deliberately no FK check

Source (external)	Bronze	Silver	Quarantined	Notes
external_position (custodian)	21	20	1	Crosswalk join + POSITION_BREAK detection: 4 rows flagged, incl. seeded FND002/AST004 conflicting-quantity case
external_cash (bank)	10	10	0	Crosswalk join + late-record flag: 1 row (CASH20260917LATE, ~21:45)
external_reference (vendor)	19	19	0	Crosswalk join + day-over-day change detection: 2 REFERENCE_BREAK rows (seeded AST005 currency flip)
external_payment (payment system)	15	15	0	Crosswalk join + exact-dup detection on payment_id (catches seeded PMT-DUP-20260917-001); all statuses loaded, SETTLED-only rule applied downstream

5.4 Incremental Processing

Two notebooks build “current state” tables via Delta MERGE upsert, and idempotency was actually verified — rerunning each notebook and confirming row counts stayed identical, not just assumed.

- 11_incremental_payment_lifecycle.ipynb — external payment events resend across the three days as their status changes (INITIATED → SETTLED, and so on), sharing a stable ID prefix with a differing suffix. A regex groups these resends into “threads,” keeping only the latest state per thread, producing silver.external_payment_current (5 threads). The pattern was inferred from a sample of only 15 rows, so it was flagged for mentor sign-off rather than treated as unquestionably correct.
- 12_incremental_position_cash_reference.ipynb — produces position_current (20 rows; 2 unresolved POSITION_BREAK keys were deliberately excluded rather than defaulted to zero, also flagged for mentor sign-off), cash_current (10 rows), and reference_current (19 rows).

5.5 Financial Calculations

13_financial_calculations.ipynb turned up two genuine bugs during development, both caught by checking the numbers against what they should mean, not just whether the code ran:

- Portfolio Value was missing quantity entirely — it summed raw benchmark prices with no unit multiplication. Fixed by joining in position_current and computing latest_close × quantity. Verification showed only one company (PORT_0003, under FUND_001) has both a benchmark price and a matching position — every other company correctly shows \$0, which is a real data-coverage fact, not a bug.
- Available Cash double-counted Capital Calls — the original formula summed capital_calls_settled + contributions – invested_capital, but capital_calls_settled was already folded into Uncalled Capital elsewhere. Fixed to available_cash_internal = contributions – invested_capital.

Output: silver.fund_financials (5 rows, fund grain) and silver.portfolio_valuation (30 rows, company grain).

5.6 Reconciliation Engine

14_reconciliation.ipynb produces silver.reconciliation (55 rows total), using one shared schema across three check types, extended with an entity_id column so POSITION and REFERENCE checks can carry an asset or company identifier — flagged for mentor sign-off, since it broadens the base reconciliation record beyond a minimal recon_id/status/difference shape.

Check	Rows	Breakdown	Notes
CASH	5	3 BREAK, 2 MISSING_EXTERNAL	\$0.01 tolerance; the join was changed from left to full outer mid-build so a fund with no external feed at all shows as MISSING_EXTERNAL rather than silently vanishing. Documented limitation: this check can never produce a MATCH by construction — internal cash is a cumulative net-capital figure, external cash is a point-in-time bank balance.
POSITION	31	2 BREAK, 1 MATCH, 29 MISSING_EXTERNAL	No independent internal position-quantity source exists, so this check does coverage-plus-internal-consistency rather than a true internal-vs-external comparison — a deliberate design adaptation, documented further in Limitations.
REFERENCE	18	1 BREAK, 17 MISSING_EXTERNAL	Same coverage-plus-consistency pattern; re-surfaces the seeded AST005 currency-flip case.

5.7 Gold Layer

Seven Gold tables, built across three notebooks, all ran successfully against real data — the seventh (market_price) sits beyond the original six-table scope, added once it became clear that Power BI's Market Price Explorer dashboard would need it.

Table	Rows	Source / Notes
gold.fund_snapshot	5	15_gold_core.ipynb — fund and fund_financials joined
gold.fund_financials	5	15_gold_core.ipynb — pass-through
gold.portfolio_valuation	30	15_gold_core.ipynb — pass-through
gold.reconciliation	55	15_gold_core.ipynb — pass-through, extended with numeric-cast columns
gold.payment_summary	—	16_gold_payments_dq.ipynb — internal/external totals per (fund_id, payment_type), tagged by source_side
gold.data_quality	—	16_gold_payments_dq.ipynb — every dq_log entry consolidated, with a computed failure_rate
gold.market_price	271	17_gold_market_price.ipynb — all 205 price bars, enriched with company/fund where a ticker matches a holding

A real design finding surfaced while building this table: six tickers, including GOOGL, map to more than one company, so joining market_price to portfolio holdings fans out — (ticker, bar_timestamp) is not a unique key. That finding directly shaped the Azure SQL design below, which uses a surrogate key for this table instead of assuming the natural key was unique.

5.8 Azure SQL Serving Layer

gold_schema_azure_sql.sql defines one schema and seven tables, matched column-for-column to the real Gold output rather than to an idealized design. Three decisions stand out: DECIMAL(18,4) is used for every money and quantity column instead of FLOAT, so aggregation stays exact; gold.data_quality's date columns are stored as NVARCHAR, because the source data genuinely stores Python .isoformat() strings — a real type mismatch traced back to its source rather than papered over; and gold.market_price uses a surrogate IDENTITY key, because of the ticker fan-out described above. Reserved words like open and close are bracketed ([open], [close]) where they appear as column names.

The load notebook, 18_load_azure_sql.ipynb, reads all seven Gold Delta tables and writes them to Azure SQL using a truncate-and-reload strategy on every run — chosen specifically because it preserves primary keys, IDENTITY columns and indexes, which a full overwrite would drop and have to recreate each time. IDENTITY columns auto-populate on the SQL Server side, and the notebook re-reads row counts after the load as a built-in verification step. Two real errors were fixed getting this notebook working: Databricks serverless compute rejects the generic jdbc data source for writes (SERVERLESS_WRITE_OPTIONS_NOT_ALLOWED), which was fixed by switching to the native sqlserver Spark connector with discrete host/port/database options instead of a raw JDBC URL; and several Gold tables inherited Bronze lineage columns (_run_id, _ingested_at) through their Silver ancestry that were never part of the Azure SQL DDL, fixed by explicitly selecting only the documented business columns before every write.

6. Data Model, Governance & Reference Data

Behind the medallion pipeline sits a smaller, less visible layer of plumbing that makes the whole thing trustworthy: the control tables that drive orchestration, the staging crosswalks that let internal and external identifiers talk to each other, and the reference data that has to exist before any reconciliation can happen at all. This is the part of the platform that a data governance reviewer would look at first.

6.1 Control & Staging Schema

The control schema holds two tables that everything else depends on. `control.etl_metadata` is the source-of-truth list Azure Data Factory's `LookupSources` activity reads on every run — one row per source, with its `landing_path`, `load_type` (FULL or INCREMENTAL) and an `is_active` flag, so adding, disabling or repathing a source is a data change, not a pipeline redeployment. `control.pipeline_audit` is the run-level ledger described in Section 4.4, written to by the `usp_AuditStart` / `usp_AuditComplete` procedures.

The staging schema holds the two crosswalk tables that make reconciliation possible in the first place: `staging.fund_id_mapping` (external fund IDs like FND001 to internal fund IDs like FUND_001) and `staging.asset_id_mapping` (external asset IDs to internal portfolio company IDs). Every Silver-layer join between an internal record and an external feed record goes through one of these two tables — without them, the platform would have no way to know that the custodian's "FND002" and the firm's own "FUND_002" are the same fund.

Database Schema & Control Framework

Control, staging and mapping tables underpinning ingestion and audit

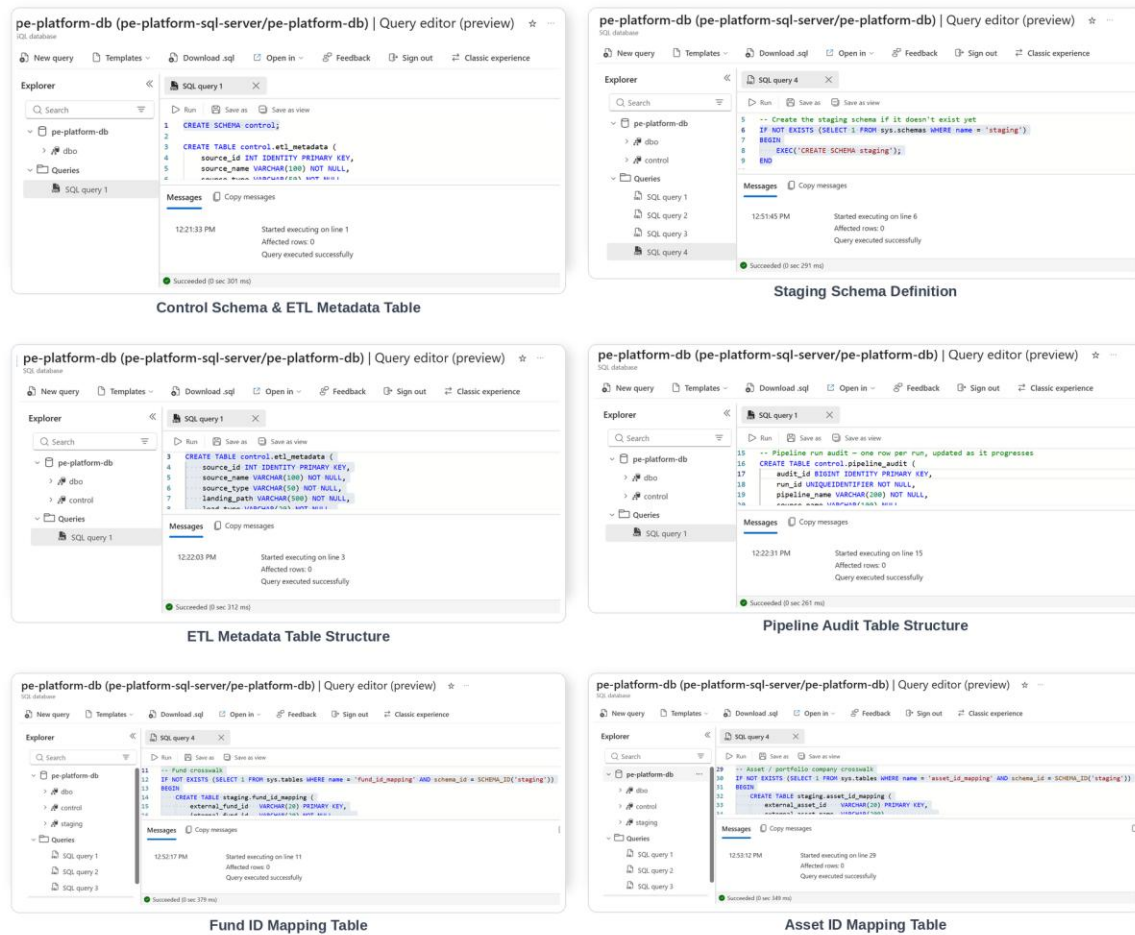


Figure 6.1 — Control schema and `etl_metadata` table definition, the staging schema being created on demand, and the `etl_metadata` / `pipeline_audit` / `fund_id_mapping` / `asset_id_mapping` table structures.

6.2 Reference Data Seeding

None of this crosswalk or metadata exists until it's seeded — and that seeding step is worth documenting in its own right, because a crosswalk with a silent gap in it produces reconciliation results that look wrong when the real problem is upstream. The fund mapping was seeded with three rows (FND001 → FUND_001, FND002 → FUND_002, FND003 → FUND_003) — deliberately leaving FUND_004 and FUND_005 unmapped, which is exactly why those two funds always show MISSING_EXTERNAL in cash reconciliation: there is no external mapping for them at any level, not just a missing cash entry for that day. The asset mapping was seeded with seven rows (AST001–AST007 to their respective portfolio companies), and `control.etl_metadata` was seeded with one row per source across all ten feeds.

Reference Data Seeding & Verification

Fund, asset and metadata mapping rows loaded and confirmed in staging

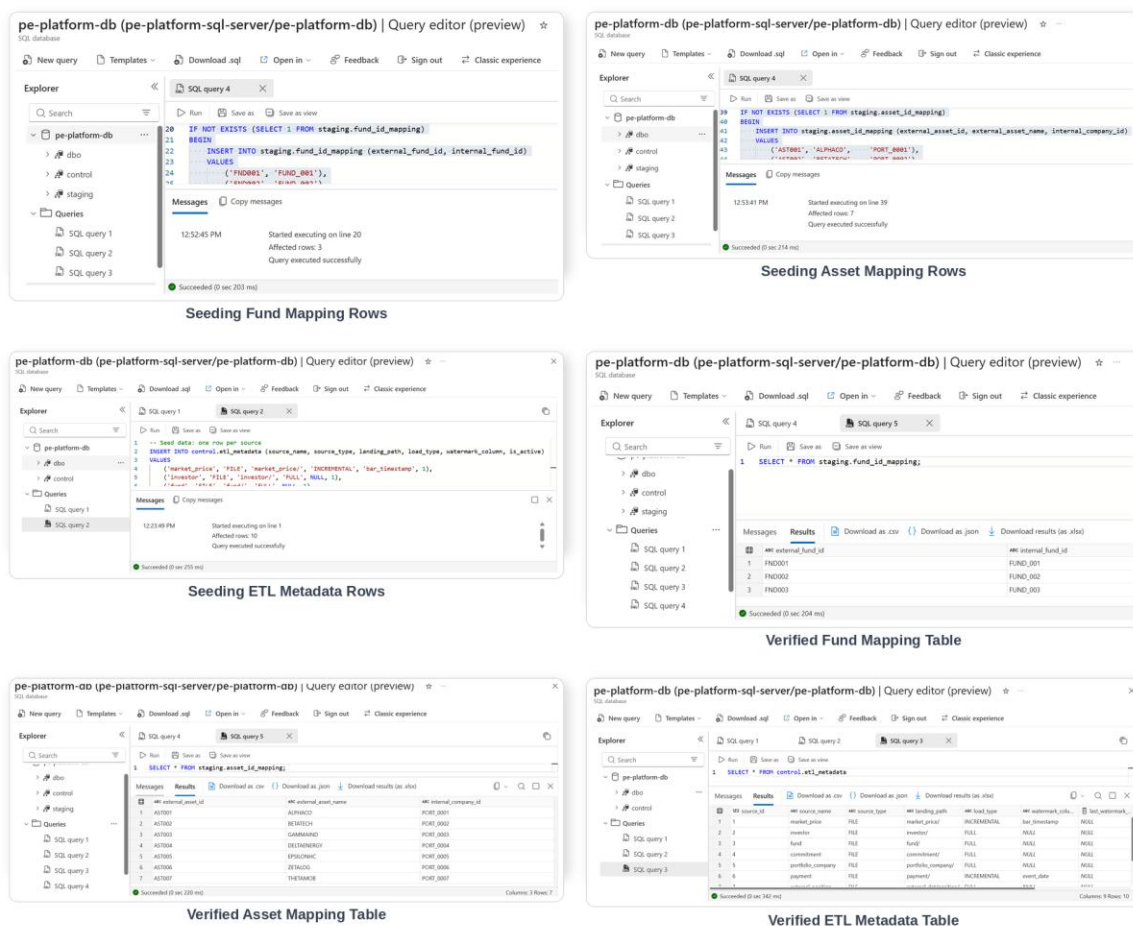


Figure 6.2 — Seeding and verifying the fund mapping, asset mapping and ETL metadata tables, confirming exactly which funds and assets have external crosswalk coverage.

This verification step is what turns “the reconciliation numbers look odd” into “the reconciliation numbers are correct, given what data actually exists” — a distinction that matters a great deal when someone downstream is deciding whether to trust a dashboard.

7. Analytics & Visualization — Power BI

The Gold layer's entire purpose is to be queried and looked at, and this section covers both halves of that: the SQL queries that prove the served data behaves the way the pipeline promised, and the Power BI dashboards built directly on top of it.

7.1 SQL Analytics Layer

A structured library of 22 queries (Q1–Q22) was written directly against the gold schema, organized into six categories — filtering and basic selects, joins, aggregations, calculated/derived metrics, window

functions, and reconciliation semantics specifically. These aren't throwaway test queries; they're the same query patterns Power BI's dashboards run underneath the hood, verified independently first.

SQL Analytics & Validation Queries

Representative Gold-layer queries: financials, payments, DQ and reconciliation

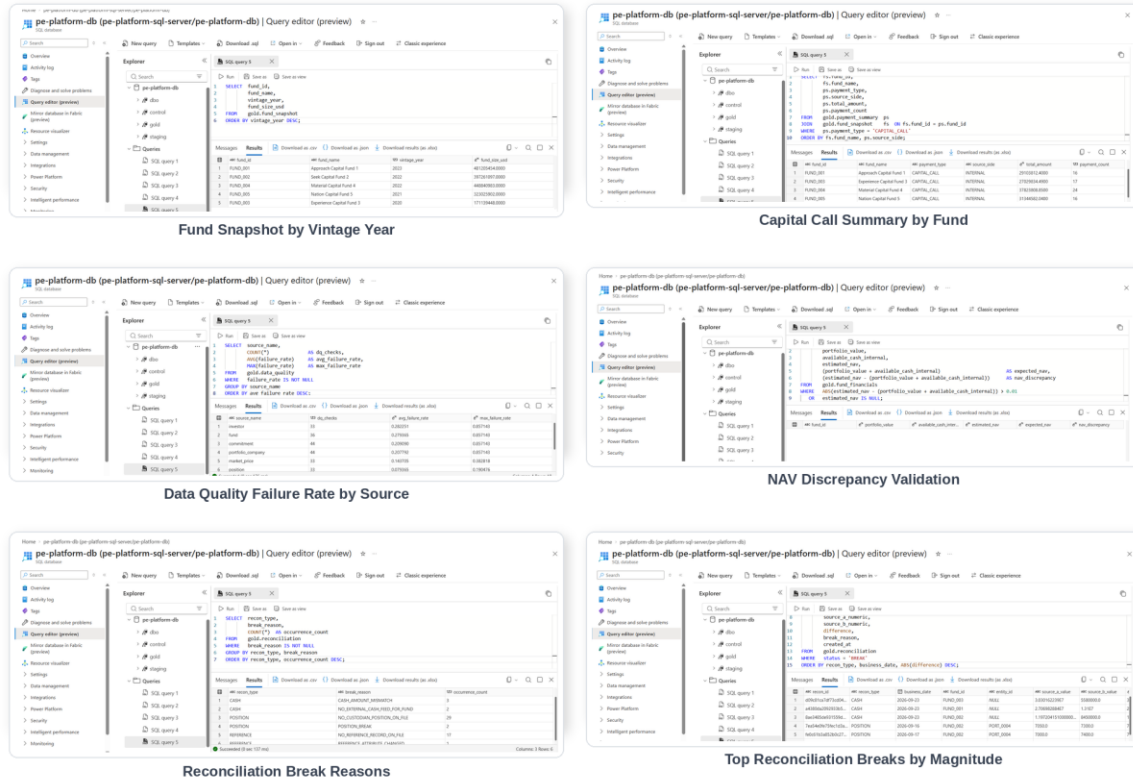


Figure 7.1 — Representative Gold-layer queries: fund snapshot ordering, capital call totals by fund, data-quality failure rate by source, NAV discrepancy validation, reconciliation break-reason frequency, and top breaks by magnitude.

The results tie back exactly to the numbers established during the Databricks build. Querying gold.reconciliation for break-reason frequency returns CASH_AMOUNT_MISMATCH (3), NO_EXTERNAL_CASH_FEED_FOR_FUND (2), NO_CUSTODIAN_POSITION_ON_FILE (29), POSITION_BREAK (2), NO_REFERENCE_RECORD_ON_FILE (17) and REFERENCE_ATTRIBUTE_CHANGED (1) — the same 55-row breakdown produced by 14_reconciliation.ipynb, now independently confirmed live in Azure SQL rather than only inside a notebook. A NAV integrity check (Q15: estimated_nav should always equal portfolio_value + available_cash_internal) and a MATCH-with-nonzero-difference check (Q16) both act as standing correctness guards — either one returning a row would mean something upstream had silently broken.

7.2 Power BI Dashboards

Six report pages connect directly to Azure SQL and cover the platform end to end, from fund-level summary down to the individual reconciliation break:

- Fund Overview — total funds, total fund size and total portfolio value at a glance, capital comparison by fund, called-vs-uncalled capital, and a commitments breakdown by fund, alongside the full fund_snapshot detail table.
- Portfolio Valuation — data coverage percentage and company count, portfolio market value by fund, and a company-by-benchmark-ticker breakdown, drawing on gold.portfolio_valuation.
- Payment Activity: Internal vs External — total payment amount and count, payment value split by type and source side, payment mix by fund, and count-vs-value by payment type, from gold.payment_summary.
- Reconciliation Breaks Scorecard — break counts by recon type and status, break reasons, and break counts by day, giving a controller the same CASH/POSITION/REFERENCE breakdown established throughout this document, now as a working scorecard rather than a notebook output.
- Data Quality Scorecard — DQ failure rate over time, an 88.24% overall pass rate, and failures broken down by both DQ rule and source table, drawing on gold.data_quality.
- Market Price Explorer — trading volume over time, price coverage by company, and a shared benchmark indicator view across tickers, drawing on the 271-row gold.market_price table.

Power BI Analytics Dashboards

Fund, valuation, payment, reconciliation, data quality and market price reporting views

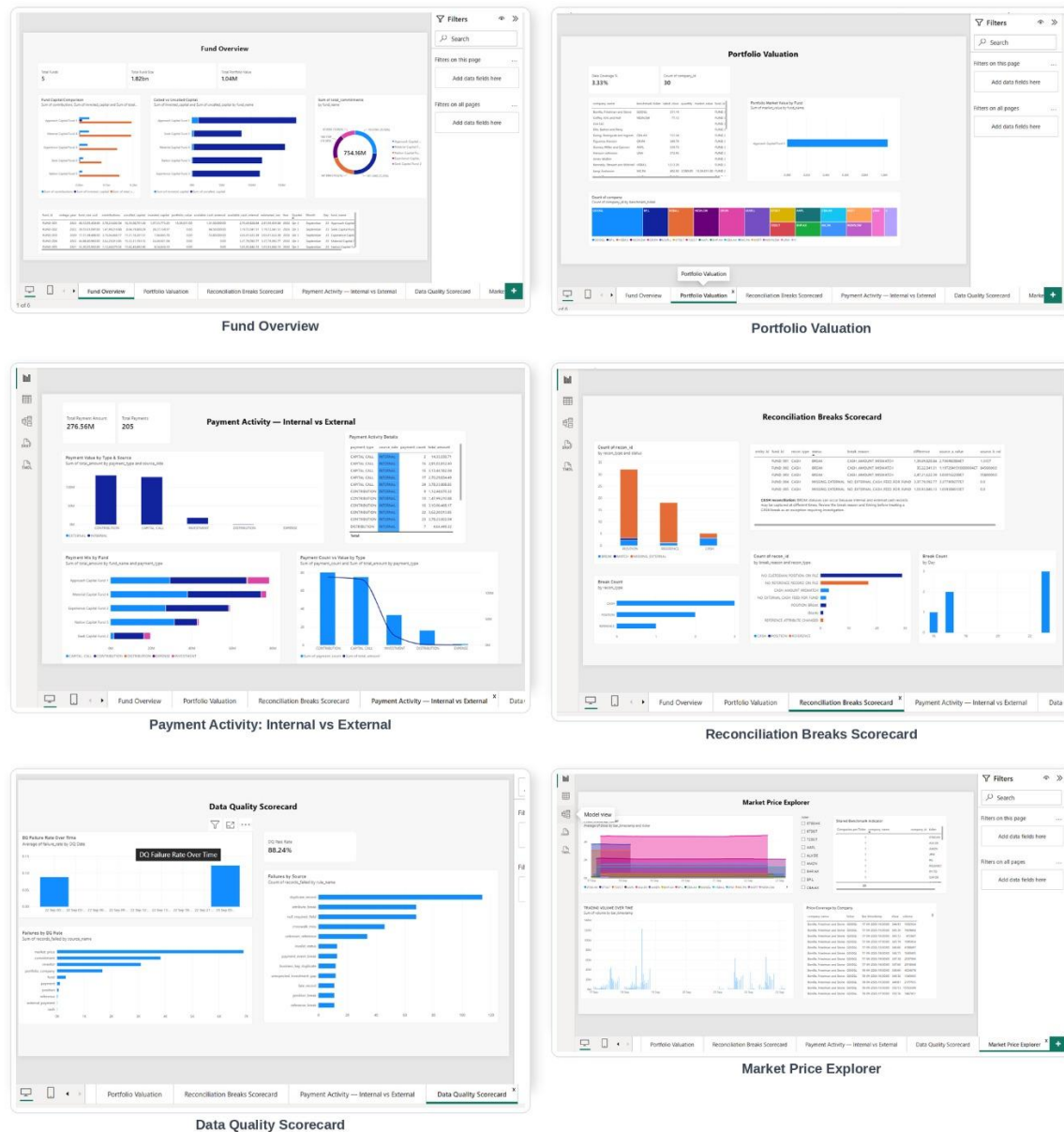


Figure 7.2 — The six Power BI report pages: Fund Overview, Portfolio Valuation, Payment Activity, Reconciliation Breaks Scorecard, Data Quality Scorecard and Market Price Explorer.

Because every one of these dashboards reads live from Azure SQL rather than from a static export, the numbers on screen are exactly the numbers produced by the Databricks pipeline and confirmed by the SQL analytics layer above — there is no separate “reporting copy” of the data that could drift out of sync with the platform underneath it.

8. Limitations

No platform this size is complete without edges. The ones below are documented on purpose — each is a specific, known boundary of the current build, not a vague disclaimer, so a reviewer or a future contributor knows exactly where to look before extending it.

- No separate frontend or backend. Power BI is the platform's interface directly, and Azure SQL Database is the sole serving layer — there is no user-facing upload portal or API tier.
- SFTP is not implemented. Custodian/bank data arrives as a pre-generated .dat feed pack uploaded once for three fixed business dates, rather than through a live SFTP connection that would need to run continuously.
- Serverless compute only. Azure Databricks runs entirely on serverless compute, with no always-on clusters — a deliberate fit to the Azure-for-Students subscription's compute-quota and time constraints, but a boundary a production deployment would need to revisit for sustained, high-volume workloads.
- Thin crosswalk coverage. The fund mapping covers only 3 of the platform's fund IDs and the asset mapping only 7 asset IDs (Section 6.2) — by design, to demonstrate the reconciliation engine's MISSING_EXTERNAL handling, but it means most of the current CASH, POSITION and REFERENCE “breaks” reflect crosswalk coverage rather than genuine external disagreement. Extending coverage is a data-seeding exercise, not a code change.
- An early credential-handling mistake was found and corrected: a Databricks setup notebook briefly referenced Azure SQL credentials directly rather than through Key Vault, before the platform's Key Vault-backed secret handling (Section 4.2) was put in place everywhere. Documented here rather than left unmentioned, since the fix is only meaningful if the original mistake is acknowledged.
- Portfolio valuation coverage is limited to one company. Only PORT_0003 (under FUND_001) has both a benchmark price and a matching position record in this dataset, so portfolio_valuation correctly shows \$0 for every other company — a data-coverage limitation of the sample data, not a calculation defect (Section 5.5).
- No DEV/SIT/UAT/PROD environment separation. The platform runs as a single environment; environment promotion and configuration-per-stage were out of scope for this build.
- The operational window is three days, not a continuous production run. That window is enough to demonstrate incremental processing, state management and reconciliation across changing data, but it is a demonstration window rather than a live, ongoing feed.

9. Conclusion

This project set out to prove that a small team, working under a genuinely tight compute and time budget, can still build every layer a real financial data platform needs. Ten sources land through an automated, audited pipeline; a medallion architecture cleans, validates and reconciles that data across three layers, with real bugs found and fixed along the way rather than a clean happy-path run; a properly designed Azure SQL schema serves the result; and Power BI turns it into something a fund operations analyst could use to catch a break the same morning it happens.

The scope adjustments made along the way — dropping SFTP for a static feed pack, skipping a user-upload portal, running everything on serverless compute — were each a considered trade against a real constraint, and none of them removed an architectural layer the original brief called for. Taken together, the eighteen-notebook pipeline, the audit and control framework, the seven-table Gold schema, and the six Power BI dashboards form a complete, working demonstration of a PE fund data platform — end to end, on real infrastructure, with real (if synthetic) breaks caught and explained rather than hidden.